

C - language

* Algorithm & Flowchart :-

Math + Electrical = Computer

program :- Set of instruction arranged in proper sequence.

Algorithm :- Mathematical way of writing the program.

algorithm & Flowchart are language independent.

The steps in the algorithm are always numbered.

1st instruction is

i) start

& ends with

End.

2) No need of define variables in algorithms.

3) Printf() / prompt are not in algorithms

4) The Instruction in algorithm are language independent.

* Flowchart :-

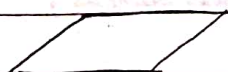
Representation of program in graphical way.

* Understand the Flow :-

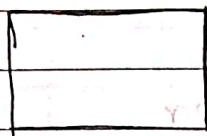
i) Symbols in Flowchart are always unidirectional (only single directional)



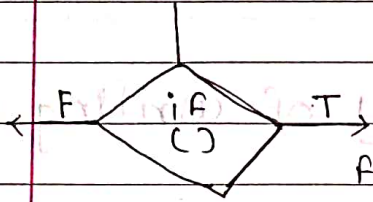
start / End



parallelogram is used for showing I/O statements.

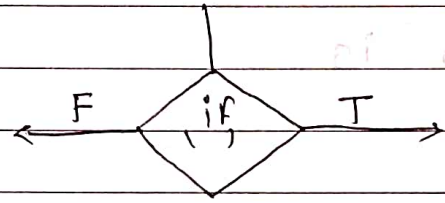


Rectangle is used for showing a processing statement.

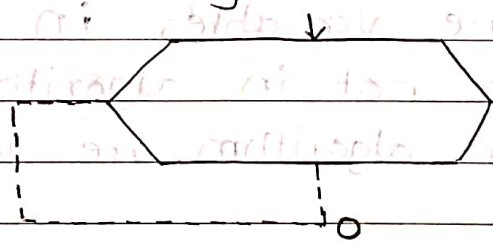


Diamond is used for if statement. It has incoming arrow one for T & one for F. It is decision making statement.

Other use of diamond is while / do-while statement.



For representing for loop use hexagon.



Iterative statement :- for / while / do-while



connectors

Every connectors is labeled A to Z. To extend the program to another ^{page} program when flow chart is too big.

Note → Symbols should be connected one another & every line should be unidirectional.

* Introduction :-

i) The C language is imperative procedural & general purpose in nature was developed by Dennis M. Ritchie at AT&T Bell Laboratories in 1972.

ii) The C language is so easy to learn & understand.

* The C character set:-

i) The C character set refers to a set of all valid characters so that use in the source program for forming words, expression & number that contains all characters.

ii) They include digits, alphabets, special symbols etc.

iii) The C language provides support for 256 characters.

iv) Digits → 0 to 9

Alphabets → A to Z a to z

Symbols → ' ~ @ ! # \$ % ^ * & [] () { }

< > + = - / \ | ; : " , . ? &

and white space.

* C keywords :-

i) keywords are predefined, reserved word used that have special meanings to compilers.

ii) keywords are parts of system & they can't be used as an identifiers.

Following is the list of keywords present in C language.

auto	double	int	struct	Total : 32 keywords in C-language.
break	else	long	switch	
case	enum	register	typedef	
char	extern	return	union	
continue	for	signed	void	
const	if	static	while	
do	goto	sizeof	volatile	
default	float	short	unsigned	

* C Identifiers :-

i) C Identifiers represent the name in the C program given to variables, function, array, structures, unions, labels etc.

ii) It can be composed of letters such as upper/lower case, underscore, digits. but the starting letter should be alphabet or an underscore.

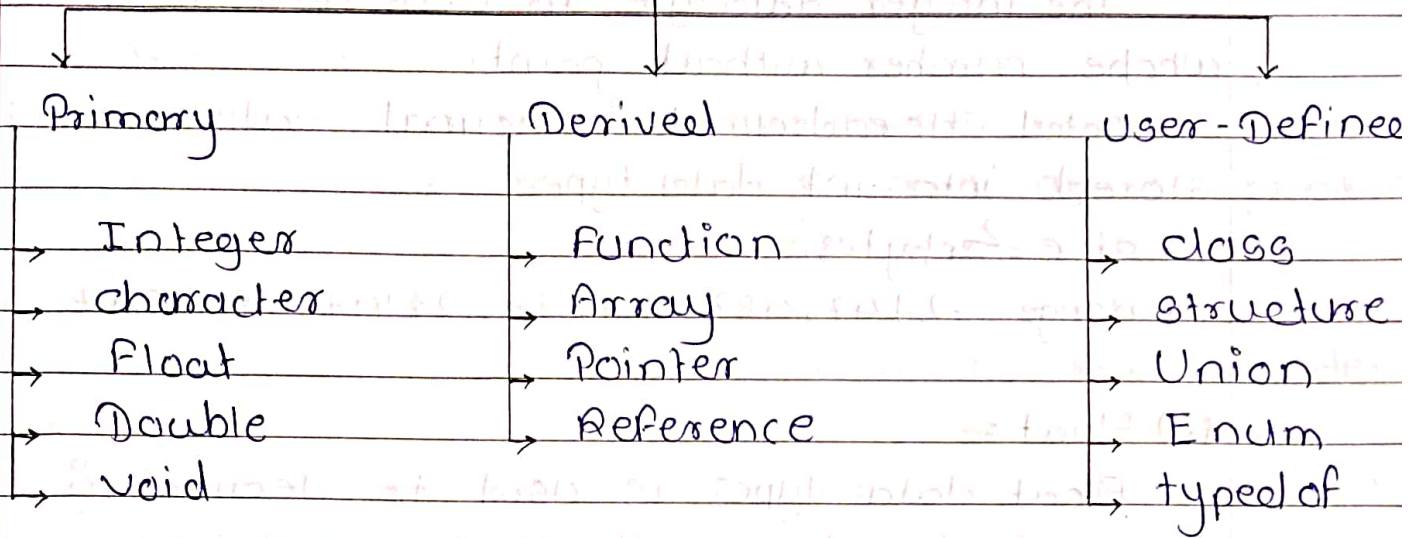
Types :- i) Internal Identifiers
 ii) External Identifiers.

Rules :-

- A valid identifier can have letters, digits & underscore.
- The first letter should be either a letter or an underscore.
- you can't use keywords as a identifiers.

* Data types.

C - Data type



i) Each variable in C has an associated data type. It specifies the type of data that variable can store like integer, character, float, double etc.

ii) Each data type requires different amount of memory and has some specific operations which can be performed over it.

A] Primary / Primitive data type :-

These are most basic data types that are used for representing simple values such as integer, float, characters etc.

B] Derived Types :-

The data types that are derived from primitive or built-in data types are referred to as derived data types.

C] User Defined Types :-

The user defined data types are defined by the user himself.

A) primary Data types :-

i) Integer :-

The integer datatype in c is used to store the whole number without point.

Octal, Hexadecimal & Decimal values can be stored into int data type.

- size = 2 bytes.

- Range :- -2,147,483,648, to 2,147,483,647.

ii) Float :-

Float data types is used to decimal & exponential values with single precision.

- Range :- $1.2E^{-38}$ to $3.4E^{+38}$

- size :- 4 bytes.

iii) Characters :-

Characters data type allows it's variable to store only a single character.

- Range = (0 to 255) or (-128 to 127)

- size = 1 byte

iv) Double :-

A Double data types is used to store decimal number with double precision & capable of holding of 64 bits of decimal no. or floating points.

- Range: 8 bytes $1.7E^{-308}$ to $1.7E^{+308}$

- size : 8 bytes

v) void :-

The void data type in c is used to specific that no value is present.

It doesn't provide result value to caller.

It has no values & no operations & is used to represent nothing.

* C variables:-

i) A variable is an entity whose values may or not may change during execution of program.

* Rules for defining variables:-

(i) A variable can have alphabets, digits & underscore.

(ii) A variable name can start with alphabets & underscore only. It can't start with digit.

iii) No whitespace is allowed within the variable name.

(iv) A variable name must not be any reserved word or keyword eg. int, float etc.

* Types of variables:-

a] Integer variable:-

i) keyword int is used to represent integer variable.

Syntax :- int list of variables;

ii) Variable should follow rules & identifiers.
 e.g. int, more, min, sum, num;

		For computer it has specific
noo		→ more address in primary memory.

C compiler use 2 bytes (16 bits) to store integer

b] Float variables:-

Float min, max, sum, num.

It requires 4 bytes & double requires 8 byte.

c] character variables:-

char min, max, sum, num.

• 1 byte required to store in turbo-C.

Q. Find the area of ^{rect} triangle.

→ #include <stdio.h>

void main()

{

int b, l, area;

clrscr();

printf("Enter the number for b & l: ");

scanf("%d %d", &b, &l);

area = b * l;

printf("Area of ^{rect} triangle : %d", area);

getch();

}

Q. Find the area of triangle:-

→ #include <stdio.h>

#include <conio.h>

void main()

{ int b, h, area; float error;

clrscr();

printf("Enter the number for b & h in");

scanf("%d %d", &b, &h);

area = $\frac{1}{2} \times b \times h$;

printf("Area of Triangle = %d in", area);

getch();

}

28/08/2023

* Constants:-

- i) constant is an entity whose value is never be changed during execution of program
- ii) Constant is always have a datatype, there is no void type constant.
- iii) we declared a constant by using:
 - a) const keyword.
 - b) #define preprocessor.

* Difference betⁿ constant & variable:

Constant	Variable.
i) A constant is a variable / value that cannot be changed once it defined.	A variable is a name associated with some memory location.
ii) It hold the fixed value which we can retrieve later but cannot change.	A variable is used to hold some value that can be changed according to the requirement.
iii) constant are generally stored in the text segment as they are read-only.	The variable are stored in a data segment, heap or stack depending on the environment it is declared in.
iv) we can only assign a value to the constant while defining it.	we can assign value to the variable anytime.
v) constant can be defined by using #define or const keyword.	variable can only be defined using the standard variable definition syntax.
e.g. #define pi 3.14	e.g. var int var = 25;
float const int pi = 3.14;	var = 10;

a) Integer constant :-

i) There is not allowed any symbol in integer constant.

e.g. const int a = \$250; — Not valid.

const int a = 250; — valid.

ii) Integer variable take 2 byte storage but Integer constant take 4 byte storage.

b) Float constant :-

i) For Float constant there is decimal point value is mandatory.

e.g. const float a = 3.14;

ii) A Float constant always stored double of float value.

∴ Float variable - 4 byte

Float constant - 8 byte.

c) Character constant :-

A character constant value must be in single quote.

e.g. const char a = 'A';

Note :-

Every character constant stored as an integer in compiler, that's way it take 2 byte storage.

d) String constant :-

A string constant value must be in double quote.

e.g. const string a = "Ram";

e7 Symbolic constant :-

We use name for value is called symbolic constant. Symbolic constant are created by using keyword "const";

e.g. `const float pi = 3.14;`

by default every symbolic constant is integer constant.

`const a = 3;`

here a is an integer constant.

* `#define pi 3.14`

There is compiler define the type of constant on value assigned to constant which is followed by `#define` preprocessor. (Preprocessor = Before the execution of program.

We can use `#define` preprocessor anywhere of the program.

`#define` does not terminated.

e.g.

`#include <stdio.h>`

`#define pi 3.14`

`void main()`
`{`

`int x = 10;`

`clrscr();`

`#define pi 3.14`

`printf("%d", pi);` print 3.14

`getch();`

`}`

b) Real arithmetical operations :-

In which both the operands will be Float & result of expression always Float.

∴ Float operator Float = Float value.
value value

c) Mixed arithmetical operations :-

In which one of operands is integer & another is ^{Float} integer be so that's way result is Float.

∴ Float operator Float = Float value
value value

B) Relational operator :-

It is also known as comparison operator. because the it is used to compare one or more operands.

i) > → greater

ii) >= → greater or equals to

iii) < → less than

iv) <= → less than or equals to

v) == → equal-equal to

vi) != → Not equal to

The result of relational operator is always writtten in integer, because it's always binary (1/0)

e.g. int a = 2, int b = 3;
printf("%d", a > b) ----- o/p = 0 - False

int c = 0
printf("%d", a < b > c) ---- o/p = 1 - true.

c] Logical operators :-

Logical operators are used for compound condⁿ. Means, if we want common result from multiple expression.

e.g.

condⁿ 1 && condⁿ 2

&& → And operator

|| → or operator

! → Not operator.

Truth Tables.

a	b	a & b	a b	a ! b	! b
0	0	0	0	1	0
0	1	0	1	1	0
1	0	0	1	0	1
1	1	1	1	0	0

a) && (And operator) :-

When the ^{all} both condⁿ are true then result is true otherwise result is false.

b) || (OR operator) :-

When minimum one ^{condⁿ} operation is true the result is true.

c) ! (Not operator) :-

It's reverse the results.

True → False

False → True.

D) Ternary operators (conditional operator)

It use (?:) symbols only.

Ternary operator checks the condition when it's true the before the ':' colon part is executed other wise after the colon is executed.

It ~~as~~ may or not to be assigned value to ~~value~~ variable.

expression ? expression 2 : expression 3 ;
condition (true) (false)

a. WAP to find largest of two numbers using Ternary operators.

```
void main ()
{
```

```
    int n1, n2, lcr;
```

```
    clrscr();
```

```
    printf("Enter the two numbers: ");
```

```
    scanf("%d %d", &n1, &n2);
```

```
    lcr = (n1 > n2) ? n1 : n2;
```

```
    printf("The largest number is %d", lcr);
```

```
    getch();
}
```

E) Assignment operator :-

Assignment operator is used to assign the value to result when left operand is equal to right operand :-

e.g. a = a + 2;

Here we use assignment operator like:

```
a += 2;
```

i) $+=$ → Addition

ii) $-=$ → subtraction

iii) $/=$ → Division

iv) $*=$ → multiplication

v) $\%=$ → modulus

e.g. void main()

{ int a=10;

clrscr();

printf("a %d\n", a);

a+=2;

printf("a %d\n", a);

getch();

}

F) Increment / Decrement Operator :-

When we want to ^{decrease} increase the value of integer variable by 1, then use increment/decrement operator ~~is~~.

Inc/Dec operator is only applied to variable (integer, float), not a constant.

$++$ → Increment

$--$ → Decrement

Here, this has two types also described.

i) post inc/dec (after the variable) → $a++$

ii) pre inc/dec (before the variable) → $++a$

e.g. of post inc/dec.

void main()

{ int a=10, b;

clrscr();

b = a++;

```
printf("a : %d , b : %d", a, b);
getch();
}
```

∴ Here First value of a used to assign to b. then add it self by 1.

a = 11, b = 10.

e.g. on pre increment.

```
void main()
{
```

```
    int a = 10, b;
```

```
    clrscr();
```

```
    b = ++a;
```

```
    printf("a : %d , b : %d", a, b);
```

```
    getch();
```

```
}
```

∴ Here the First is increment in a by 1 then add to be.

∴ a = 11; b = 11;

G] Bitwise operator :- &, |, ^

Bitwise operator is very complex to understand. Their are three operator.

i) & → and = all condⁿ are true = true

ii) | → or = at least one condⁿ is true = true

iii) ^ → xor = when the both the are different then output true.

i) & bitwise operator :-

e.g. int a = 10, b = 8;

then c = a & b = ?

weight of bytes, ... 16 8 4 2 1

For $a = 10$

For $b = 8$

For $c = a \& b$

then $c = 8$;

0	1	0	1	0
0	1	0	0	0
0	1	0	0	0

ii) Bitwise or :-

e.g. $\text{int } a = 10 ; b = 8 ;$

then $c = a | b ?$

→

weight of bytes, ... 16 8 4 2 1

For $a = 10$

For $b = 8$

then $c = a | b$

$c = 10 ;$

0	1	0	1	0
0	1	0	0	0
0	1	0	1	0

iii) xor Bitwise :-

e.g. $\text{int } a = 10 ; b = 8 ;$

then $c = a \wedge b ?$

→

weight of bytes, ... 16 8 4 2 1

For $a = 10$

For $b = 8$

then $c = a \wedge b$

$c = 2$

0	1	0	1	0
0	1	0	0	0
0	0	0	1	0

IMP

Q. WAP to test whether the given number is odd or even by bitwise & ternary operator.

```

→ #include <conio.h>
#include <stdio.h>
void main()
{
    int a = 10;
    clrscr();

    (a & 1) ? printf("odd no.") : printf("even no.");
    getch();
}

```

Q. WAP to test whether the given number is odd or even by using XOR operator.

```

→ #include <conio.h>
#include <stdio.h>
void main()
{
    int a = 11;
    clrscr();

    (a ^ 1) == (a + 1) ? printf("even no.") : printf("odd no.");
    getch();
}

```

$$\begin{array}{r}
 \begin{array}{cccc}
 & 8 & 4 & 2 & 1 \\
 a & 1 & 0 & 1 & 1 \\
 a^1 & 1 & 0 & 1 & 1 \\
 12 & 1 & 1 & 0 & 0
 \end{array} \\
 \hline
 \begin{array}{cccc}
 & 8 & 4 & 2 & 1 \\
 & 1 & 1 & 0 & 0 \\
 & 1 & 1 & 0 & 0
 \end{array}
 \end{array}$$

Date: 30/08/2023

H7

Shift operator :-

i) Shift operator is only used for integer data type variables for moving to left / right side it.

a) $\gg$ $\longrightarrow$ right shift

b) $\ll$ $\longrightarrow$ left shift

e.g. of right shift

$a = 10;$

Byte weightage :- 32 16 8 4 2 1

$a = 10$ 0 0 1 0 1 0

If $a \ll 2$

then add two zero value at last position & shifted starting of two zero.

$\therefore$ 1 0 1 0 0 0 $\leftarrow$ added.

$a \ll$

$a \ll 2 = 40$

If $a \gg 2$

then 0 0 1 0 1 0 $\leftarrow$ shifted.

$a \gg 2 = 2$

i] Special operators :-

a] Comma operators :-

It is used grouping of some element which homogenous in nature.

Ex. WAP to swap values of two variables :-

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int a, b, x, t;
```

```
    clrscr();
```

```
    printf("Enter two numbers :");
```

```
    scanf("%d%d", &a, &b);
```

```
    t = a, a = b, b = t; // use of comma operator
```

```
    printf("Now a = %d", a);
```

```
    printf("Now b = %d", b);
```

```
    getch();
```

```
}
```

IMP → Some swapping techniques :-

i) $a = a + b$ ii) $a = a * b$ iii) $a = a ^ b$

$b = a - b$ $b = a / b$ $b = a ^ b$

$a = a - b$ $a = a / b$ $a = a ^ b$

b] sizeof operator :-

i) Size of operator is ~~work~~ look as function but it's work as a operator.

ii) sizeof operator is return the integer size of storage of variable/constant.

Some important example of sizeof operator-

i)

`sizeof(3.14)`

∴ It is float constant ∴ then it's return 8
 For it's byte value.

ii)

`int a = 30;`

`sizeof(a)`

∴ It return 2 because it is an integer type value.

iii)

`sizeof(30);`

It return 4 because it is an integer constant.

iv)

`int a = 10, b = 20;`

`sizeof(a + b);`

∴ Here the sizeof operator does not solve the expression, it's only return value, it ~~is~~ 2 because both are integer.

* Expression :-

- i) Expression is collection of operators & operands.
- ii) Expression have at least one any type of operator.
- iii) Expression may have one or more one operand which is operators performs operations.

iv) Priority of executing of expression is Bodmas Rule

i) Solve bracket first.

ii) / , %

iii) *

iv) +

v) -

e.g. $a = 12 * (4/5)$

$= 12 * 0$ $\therefore$ Here division operator is used then zero.

$a = 0$

$a = 12 + (4/5)$

$a = 12 + 0$

IMP Type casting (Type conversion):-

Type casting is a process in which we convert one datatype to another datatype.

Type casting have two types.

i) Implicit casting

ii) Explicit casting.

i] Implicit casting:-

The casting which is done by compiler automatically during the execution of program.

e.g. $3 + 5.5$

$3.0 + 5.5$

8.5

8.5

Here we get two o/p. when user want o/p in integer format with by assign the value then output is 2000, otherwise o/p is 8.5.

ii) Char c;

$c = 'c' + 1$

Here compiler automatically gets ASCII code of c. its go then in which add 1.

$\therefore$ 99 & 99 (is the ASCII code of D).

Then $c = D$

ii) Explicit casting :-

i) In Explicit casting, we force/applied casting before the execution of program using the type cast operator, $[()]$.

Eg.

~~int a = (int) 5 + 3.5;~~

i) `int a = (int) 3.5 + 5;`

`int q = 3 + 5;`

`a = 8;`

Here we use integer type casting operator

ii) i.e. (int) .

ii) `int a = int(3 + 5.5);`

`a = 8;`

here the o/p is same but procedure is different.

* I/O Statement :-

A) `printf()`;

i) `printf` statements contains two parts.

a) string part.

b) variable part.

ii) `printf` statement return the How much columns required for display the string in integer format.

eg. `a = printf("%d", 1234)`

o/p = `a = 4;`

Now `printf("%d", printf("%d", 1234));`

∴ Here the compiler executes inner `printf()` which o/p is 4234, then return the column size is 4

then final i/o is 12345.

B) scanf():

- i) It is used to get value from user.
- ii) It get only integer, float & character inputs
- iii) scanf() contain two parts
 - a) control string → format specifier contains
 %d = integer, %ld = long integer
 %f = float, %lf = double float.

b) variable list :- in which we ge stored the values (input) from user.

- iv) scanf() return the integer value which is corresponding to values read by user.
- v) scanf() use for validation also.

eg. void main()

```

{
    int a, b, c, sum;
    clrscr();
    printf("Enter the three values");
    t = scanf("%d%d%d", &a, &b, &c);

    if (t != 3)
    {
        printf("In one or more than one input  
is case incorrect!");
        getch();
        return 0; // exit(0);
    }

    sum = a + b + c;
  
```

```
printf("sum = %d", sum);  

getch();  

return(0);  

}
```

// in c use return() or exit() for the logical end which is appears multiple times

Q. why return 0 in c?

→ As assumption 0 is good return value in c. but we can use any integer value.

∴ return 0 ✓
 return 1 ✓
 return -1 ✓

05/09/2023

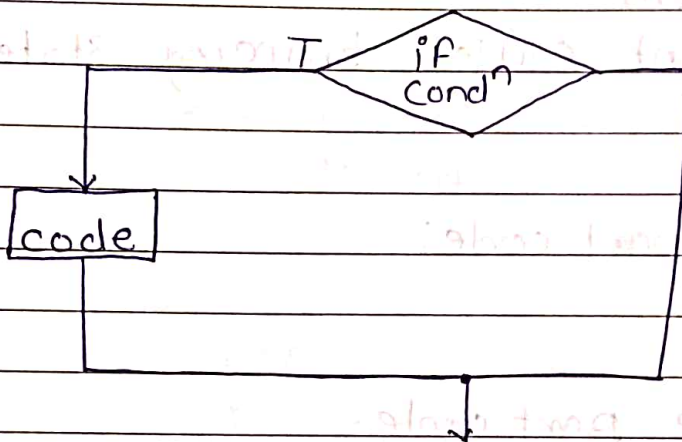
Unit-II

i) Simple IF statement :-

There is only one part i.e. if condition block is executed when condition is true otherwise program continues from next step.

Syntax :-

```
if (condn) { }
```



∴ IF the if statement have non-zero value then its true condition.

e.g. `if(-1)` — True

`if(1)` — True

`if(0)` — False

Q WAP to find smallest from three numbers using simple IF statement.

```
→ void main()
{
```

```
    int a, b, c, small;
```

```
    clrscr();
```

```
    printf("Enter the three numbers : ");
```

```
    scanf("%d%d%d", &a, &b, &c);
```

```
    small = a;
```

```
if (small > b)    small = b;
if (small > b)    small = c;
```

```
printf("In small number = %d", small);
getch();
}
```

ii) if-else statement:-

This statement called branching statement

e.g.

```
if (cond? 1)
{
    True part code;
}
else
{
    False part code.
}
```

iii) Nested if statements:-

Syntax:-

```
if (cond? 1)
{
    // if (cond? 2)
    {
    }
    else
    {
    }
}
else
{
    if (cond? 3)
    {
    }
}
```

```

else
{
}
}

```

iv) else if ladder:-

Syntax:-

```

if (cond1)
{
    #code
}
else if (cond2)
{
    #code
}
:
else
{
}

```

v) switch case statements:-

Syntax-

```

switch ( Expression ( integer / float / character ) )
{
    case n1:
        //code
        break;
    :
    default:
        code
}

```

* Looping statement :-

Loops in C is used to execute the block of code several times according to the condition given in the loop.

Types of loops :-

- i) For statement.
- ii) do-while statement
- iii) while statement.

i) For Loop :-

It execute the code until condition is False.

There is three parameters in loop statement.

- a) Initialization
- b) Condition
- c) Increment / Decrement.

Syntax :-

```
for( initialization; condition; inc/dec)
{
    //Code to be executed
}
```

The for loop also called per-tested loop. It is better to use for loop if the number of iteration is known in advanced.

Q. WAP to print Fibonacci Series with n numbers.

→ #include <stdio.h>

#include <conio.h>

void main()

{

int a, b, c, n; n = 3;

clrscr();

```
printf("Enter the total numbers : ");
scanf("%d", &n);
a=0;
b=1;
printf("%d ln %d", a, b);
```

```
for( ; i <= n ; ++i)
{
    c=a+b;
    printf("ln %d", c);
    a=b;
    b=c;
}
```

```
}
getch();
```

Q. WAP to add odd numbers, even numbers sep Separately from the range of number 1 to 50.

→

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int i, odd=0, even=0;
    clrscr();
    for(i=1; i<=50; i++)
    {
        (i%2==0)? even+=i : odd+=i;
    }
```

```
printf("In Even numbers sum %d", even);
printf("In Odd numbers sum %d", odd);

getch();
}
```

Q. WAP to Find Factorial of the number :

```
→ #include <stdio.h>
#include <conio.h>

void main()
{
    int n, fact = 1;

    clrscr();

    printf("Enter number : ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        fact *= i;
    }

    printf("In Factorial is : %d", fact);

    getch();
}
```

Q. Write a program to find largest number among the 'n' non-zero positive integers.

```
→ #include <stdio.h>
#include <conio.h>

void main()
{
    int n, max;
```

```
int n, no, lar=0;
clrscr();
printf("Enter the size : ");
scanf("%d", &n);
```

```
for(; n>0; n--)
{
  printf("Enter the number : ");
  scanf("%d", &no);
```

```
  if(lar<no)
    lar=no;
```

```
}
printf("In largest number : %d", lar);
```

```
getch();
}
```

Q. write the program to test the number is prime or not?

```
① → int main()
{
  int no, i, Flag=1;
  clrscr();
  printf("Enter the number : ");
  scanf("%d", &no);
```

```
  for(i=2; i<=no/2; i++)
  {
    if(no%i==0)
    {
      Flag=0;
      break;
    }
  }
}
```

```
if(Flag==0)
```

```
printf("%d is NOT prime number :", no);
```

```
else
```

```
printf("%d is Prime number : ", no);
```

```
return(0);
```

```
getch();
```

```
}
```

ii) By counter method.

```
int main()
```

```
{
```

```
int i, no, count=0;
```

```
clrscr();
```

```
printf("Enter the numbers : ");
```

```
scanf("%d", &no);
```

```
for(i=2; i<=no/2; i++)
```

```
{ if(no%i==0)
```

```
count++;
```

```
break; }
```

```
}
```

```
if(count!=0) printf("Not Prime number ");
```

```
printf("Prime number ");
```

```
getch();
```

```
return 0;
```

```
}
```

iii)

```
int main()
{
    int i, no;
    clrscr();
    printf("Enter the numbers : ");
    scanf("%d", &no);

    for(i=2; i<=no/2; i++)
    {
        if(no%i==0)
        {
            printf("Not prime ");
            getch();
            return 0;
        }
    }

    printf("Prime number");
    getch();
    return 0;
}
```

Q. Print Find sum where $\text{sum} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$

→ int main()

```
{
    int i, n;
    float sum = 0;
    clrscr();
    printf("Enter the number : ");
    scanf("%d", &n);
```

```
for( ; n > 0; n--)
```

```
    sum = sum + 1.0/n;
```

```
printf("Total = %d", sum);
```

```
getch();
```

```
return 0;
```

```
}
```

Q WAP to print common divisor of 2 non-zero positive integer numbers-

→ #include <stdio.h>

#include <conio.h>

void main()

{

int n1, n2, i;

clrscr();

printf("Enter two numbers : ");

scanf("%d %d", &n1, &n2);

if(n1 > n2) a = a ^ b, b = a ^ b, a = a ^ b;

for(i = 1; i <= n1/2; i++)

{

if(n1 % i == 0 && n2 % i == 0)

printf("In %d", i);

}

getch();

}

Q WAP to evaluate a series $S = 1/1! + 2/2! + \dots + n/n!$

→ #include <stdio.h>

#include <conio.h>

```

void main()
{
    int n, i;
    float sum = 0, f = 1;
    clrscr();
    printf("Enter the size of series : ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        f = f * i;
        sum = sum + i / f;
    }
    printf("In sum = %.f", sum);
    getch();
}
    
```

* while statement =

i) It is working similar to for loop.
 ii) when we want to execute the specific code with particular condition but we don't have idea about how much iteration required for it then we use while loop.

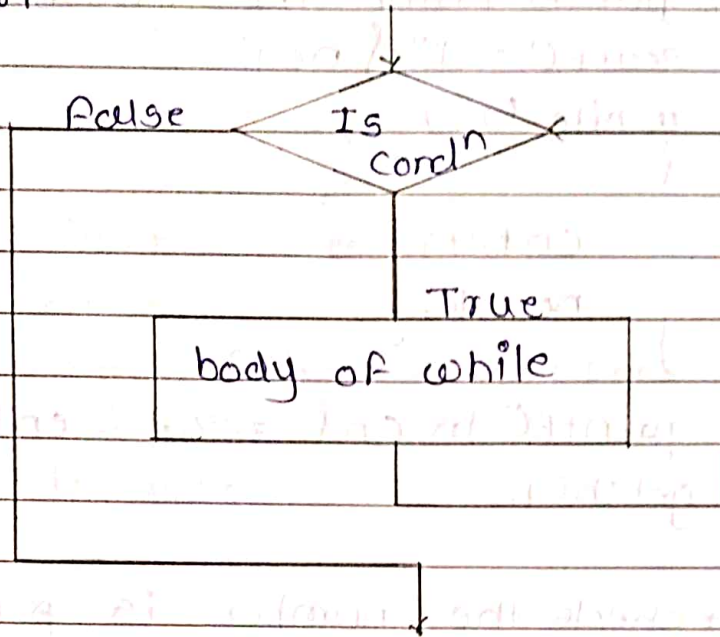
iii) In while loop, first condition is checked the block of code is executed until the condition is false.

Syntax :-

```

while(condition)
{
    // code
    update the value of control variable
}
    
```

Flow chart:-



Q. WAP to print number from 1 to 100 which are divisible by 4 & 6;

→ void main()

```
{
    int i=1;
```

```
    clrscr();
```

```
    for(while(1=100))
    {
```

```
        if(i%4==0 && i%6==0)
```

```
            printf("%d", i);
```

```
            i++;
```

```
        }
```

```
        getch();
```

```
    }
```

Q. WAP to count digits in a zero number.

→ void main()

```
{
    int no, cnt=0;
```

```
    clrscr();
```

```

printf("Enter the number : ");
scanf("%d", &no);
while (no)
{
    cnt++;
    no /= 10;
}
printf("In cnt = %d", cnt);
getch();
}

```

Q. WAP to check the number is Armstrong or not
 → void main()

```

int no, temp, sum=0, r;
clrscr();
printf("Enter the number : ");
scanf("%d", &no);
temp = no;
while (no)
{
    r = no % 10;
    sum = sum + r * r * r;
    no /= 10;
}
if (sum == temp)
    printf("Enter the Armstrong number : ")
else
    printf("Not armstrong number");
getch();
}

```

Q. WAP to check whether the given number is pollindrom or not.

→ void main()

{

int no, r, temp, rev = 0;

clrscr();

printf("Enter the number : ");

scanf("%d", &no);

temp = no;

while(no)

{

r = no % 10;

rev = (rev * 10) + r;

no /= 10;

}

if (rev == temp)

printf("Pollindrom number");

else

printf("Not pollindrom number");

getch();

}

* do-while loop :-

i) When we want the block of code executed at least ones without condⁿ then we use do-while in there.

ii) Do while loop terminated with semicolon.

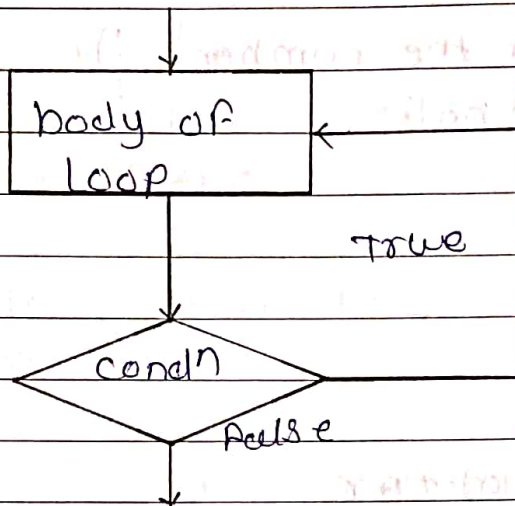
iii) Flow →

body → condition.

Syntax :-

```
do
{
    //code
}while(condition);
```

Flow chart



* Nested For loop

Q. Write the program that multiplication tables for number 1 to 10.

→ void main()

```
{
    int i, j;
```

```
    clrscr();
```

```
    for(i=1; i<=10; i++)
```

```
{
```

```
        for(j=1; j<=10; j++)
```

```
{
```

```
            printf("%4d", i*j);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    getch();
```

```
}
```

Q. write a program to demonstrate use of continue statement.

```

→ void main()
{
    int i;
    clrscr();
    for(i=0; i<=10; i++)
    {
        printf("%4d", i);
        continue;
        printf("%4d", i*5);
    }
    getch();
}

```

* Break Statement -

i) break statement use for terminating looping structure.

* Drawback -

ii) C is structured programming language but goto make it unstructured.

iii) take more to execute.

iii) Goto statement have a label.

```

Q. void main()
{
    int i=1;
    clrscr();
Next: if(i<=10)
    {
        printf("%d", i);
        i++;
        goto Next;
    }
    getch();
}

```

Q. Label below the goto statement.

```

→ void main()
{
    int i=0;
    clrscr();

xyz: if(i>9) goto abc;
    i++;
    printf("%d\t", i);
    goto xyz;
abc: getch();
}

```

Q. Wap to add digits of number using PP & goto statement.

```

→ void main()
{
    int no, sum=0;
    clrscr();
    printf("Enter the number : ");
    scanf("%d", &no);

    Label: sum = sum + no % 10;
    if(no)
    {
        no /= 10;
        goto Label;
    }

    printf("sum of digits = %d", sum);
    getch();
}

```

* Jumping in & jumping out from the looping statement

* Array:-

Array is collection of homogeneous element which ^{are} stored in continuous memory location.

• Types of Array:

i) Single dimensional array:

ii) Multi-dimensional array (2-d array):

* Single (one) dimensional array:-

When the array elements are stored in one row with continuous memory location.

e.g.

```
int num[10]
```

1000 →	num[0]
1002 →	num[1]
1004	⋮
⋮	⋮
1018	num[9]

* Initialization 1D array:-

i) Compile Time

ii) Run Time

• Compile Time - also called static initialization.

In which we initialize all the value to array at compile Time.

e.g. `int array[5] = { 1, 2, 3, 4, 5 };`

Imp → i) If `int num[10] = {0};`
 Here all the values of ^{array} elements are zero.

ii) If `int num[10] = {1, 2};`
 then only 1, 2 are the first & second element of the array respectively. Others are zero.

iii) `int num[] = {1, 2, 3, 4};`
 Here compiler automatically assign size of array as 4.

iv) If `int num[] = {0, 0, 0, 0};` then compiler wants writes zero upto it's size.

• Run-Time initialization / dynamic initialization.

`int array[10];`

a. write a program to find smallest number from array with size 10.

→ `void main()`
`{`

`int num[10] = {10, 5, 8, 11, 20, 40, 70, 90, 1, 2};`

`int i, small;`

`clrscr();`

`small = num[0];`

`for(i = 1; i < 10; i++)`

`{`

`if (small > num[i])`

`small = num[i];`

`}`

`printf("Enter the number : %d", small);`

`getch();`

`}`

Q. WAP to create an array of 10 element & count odd & even number from it

→ void main()

{

int no[10], even=0, odd=0;

clrscr();

printf("Enter 10 elements of an array:");

for(i=0; i<10; i++)

{

scanf("%d", &no[i]);

(no[i] % 2 == 0) ? even++ : odd++;

}

printf("Total even numbers : %d", even);

scanf("Total odd numbers : %d", odd);

getch();

}

Q. To find largest number from 2d array of size 3x3

→ void main()

{ int a[3][3] = { 2, 1, 10, 11, 3, 6, 7, 8, 9 };

clrscr();

int i, j, largest = a[0][0];

for(i=0; i<3; i++)

{

for(j=0; j<3; j++)

{

if(largest < a[i][j])

{ largest = a[i][j];

}

}

}

printf("largest number : %d", largest);

getch();

}

* Multiplication of Matrix :-

$$a = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 1 & 2 \\ 2 & 3 & 1 \end{bmatrix} \quad b = \begin{bmatrix} 3 & 2 & 1 \\ 1 & 3 & 2 \\ 2 & 1 & 3 \end{bmatrix}$$

$$a * b = \begin{bmatrix} 3+2+6 & 2+6+3 & 1+2+3 \\ 9+1+4 & 6+3+2 & 3+2+6 \\ 6+3+2 & 4+9+1 & 2+6+3 \end{bmatrix} = \begin{bmatrix} 11 & 11 & 6 \\ 14 & 11 & 11 \\ 11 & 14 & 11 \end{bmatrix}$$

* void main()

{

int a[3][3], b[3][3], i, j, k, sum;

char c;

printf("Enter elements in First matrix");

for(i=0; i<3; i++)

{

for(j=0; j<3; j++)

{

scanf("%d", &a[3][3]);

}

}

printf("Enter elements in Second matrix");

for(i=0; i<3; i++)

{

for(j=0; j<3; j++)

{

scanf("%d", &b[3][3]);

}

}

```

printf("multiplication of matrices : \n");
for (k=0; k<3; k++)
{
    for (i=0; i<3; i++)
    {
        sum=0;
        for (j=0; j<3; j++)
        {
            sum=sum+ a[k][i]* b[j][i];
        }
        printf("%d\t", sum);
    }
}
getch();

```

* Area of circle - πr^2

* Area of rectangle = $w \times l$

* Volume of cube = a^3

* Strong num = number whose sum of the factorial of digit & of digit is equal to original number.

Q.1 wap to input the 3 sides of a triangle & print it's corresponding type-

Equilateral - All sides are equal.

Isoceles - 2 equal sides.

Scalene - no equal sides.

```

→ #include <stdio.h>
#include <conio.h>
void main()
{
    int a, b, c;
    clrscr();
    printf("Enter the size of triangle - ");
    scanf("%d %d %d", &a, &b, &c);

    if (a == b && a == c)
        printf("Equilateral Triangles");
    else if (a == b || a == c || b == c)
        printf("Isoceles Triangles");
    else
        printf("Scalene Triangle");

    getch();
}
    
```

Q.2 pri wap to print this type of pyramid-

```

      *
     **
    ***
   ****
  *****
 *****
  
```

```
#include <stdio.h>
```

```
int main()
```

```
{
    int n, row, c, i, temp;
```

```
    printf("Enter No. of rows - ");
```

```
    scanf("%d", &n);
```

```
    temp = n;
```

```
    for (row = 1; row <= n; row++)
    {
```

```
        for (i = 1; i <= temp; i++)
```

```
        {
            printf(" ");
```

```
            temp--;
```

```
        }
```

```
        for (c = 1; c <= 2 * row - 1; c++)
```

```
        {
```

```
            printf(" * ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Q. WAP to evaluate series-

i) $p = (1 * 2) + (2 * 3) + \dots + (9 * 10);$

→ #include <stdio.h>

```
int main()
```

```
{
    int i = 1;
```

```
    int sum = 0;
```

```
    for (; i <= 10; i++)
```

```
    {
```

```
        sum = sum + (i * (i + 1));
```

```
    } printf("sum = %d", sum);
```

```
    return 0;
```

```
}
```

- ② $a = 1/2 + 3/4 + 5/6 + \dots + 13/14$
- ③ $S = 2/5 + 5/9 + 8/13 + \dots + n$
- ④ $a = x + x^2 + x^3 + x^4 + \dots + x^{10}$
- ⑤ $p = x + x^3/3 + x^5/5 + x^7/7 + \dots + n$
- ⑥ $S = (13 \times 1) + (12 \times 2) + \dots + (1 \times 13)$
- ⑦ $S = 1 + 1/(2^2) + 1/3^3 + 1/4^4 + \dots + 1/5^5$

- ① $Q = 1/2 + 3/4 + 5/6 + \dots + 13/14 - Q = ?$
- ② $S = 2/5 + 5/9 + 8/13 \dots n$
- ③ $S = x + x^2 + x^4 + \dots + x^{10}$
- ④ $P = x + x^3/3 + x^5/5 + x^7/7 + \dots + n$
- ⑤ $S = (13 \times 1) + (12 \times 2) + \dots + (1 \times 13)$
- ⑥ $S = 1 + 1/(2^2) + 1/3^3 + 1/4^4 + 1/5^5 \dots$

① $Q = 1/2 + 3/4 + 5/6 + \dots + 13/14$
→

```
void main()
```

```
{
    int i, j; float sum = 0.0;
    clrscr();
```

```
    for(i=1; j=2; i<=13; i+=2; j+=2)
    {
```

```
        sum = (float) i/j + sum;
    }
```

```
    printf("%f", sum);
```

```
    getch();
}
```

② void main :

```
{
    int n, i, j = 5;
```

```
    float sum = 0.0;
```

```
    clrscr();
```

```
    printf("Enter limit : ");
```

```
    scanf("%d", &n);
```

```

for ( i = 1; i <= n; i++)
{
    sum = (float) i / j + sum;
    j = j + 4;
    sum = (float) (i+1) / j + sum;
    j = j + 4;
}
printf(" sum %.f", sum);
getch();
}
    
```

③ $S = x + x^2 + x^4 + \dots + x^{10}$

→ #include <math.h>

void main()

```

{
    int n, i, sum = 0;
    
```

```

clrscr();
    
```

```

printf("Enter an number:");
scanf("%d", &n);
    
```

```

for ( i = 1; i <= 10; i++)
{
    
```

```

        sum = sum + pow(n, i);
    }
    
```

```

printf("sum = %d", sum);
getch();
}
    
```

④ $S = (13 \times 1) + (12 \times 2) \dots + (1 \times 13);$
 → void main()
 {
 int i = 1, j = 13, sum = 0;
 clrscr();
 for(; i <= 13 ; i++)
 {
 sum = sum + (i * j);
 j--;
 }
 printf(" %d", sum);
 getch();
 }

Q. write a program to copy content of one file to another file.

```

→ #include <stdio.h>
#include <conio.h>
void main()
{
    FILE *fptr1, fptr2;
    char c;
    clrscr();
    fptr1 = fopen("Filename1.txt", "r");
    if (fptr1 == NULL)
    {
        printf("ERROR");
        exit();
    }
    fptr2 = fopen("Filename2.txt", "w");
    if (fptr2 == NULL)
    {
        printf("ERROR");
        exit(1);
    }
    while ((c = getc(fptr1)) != EOF)
    {
        putc(c, fptr2);
    }
    fclose(fptr1);
    fclose(fptr2);
    getch();
}

```